SplitSnap: System Architecture & Technical Report

End-to-End Vision-Language Receipt Parsing & Mathematically Reconciled Proportional Bill Allocation

Project: Track 1 Challenge Submission
Authors: CrackHeads Team
Repository: Navdeep2032/CrackHeads
Date: October 2026

1. Executive Summary & Problem Context

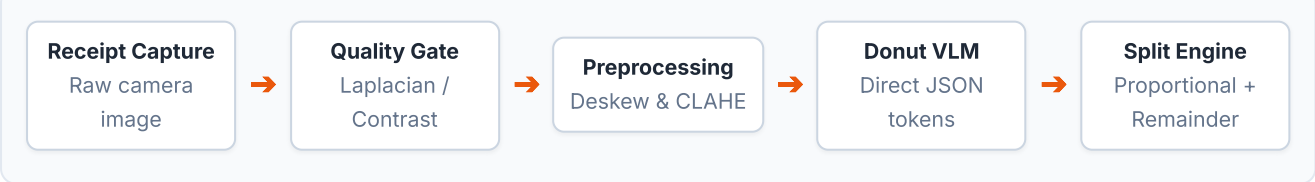
Dining bill splitting has traditionally relied either on crude total division (unfair to asymmetric consumers) or laborious manual item transcription into mobile calculators and chat apps. The technical complexity escalates sharply in the context of **Indian restaurant receipts**, characterized by multi-tiered tax structures (CGST, SGST, IGST), optional or compulsory service charges, varying item-level discounts, mixed English/Hindi vocabulary, and variable thermal print qualities.

SplitSnap resolves this via a unified pipeline pairing an OCR-free vision-language transformer (Donut) with a deterministic arithmetic reconciliation and weighted assignment engine. Key achievements include:

- **Direct Image-to-Structured JSON:** Replaces fragile two-stage pipelines (OCR bounding-box detection + text heuristic grouping) with an end-to-end encoder-decoder model.
- **Arithmetic Self-Consistency Verification:** Automatically checks whether line items sum to subtotal, verifies tax computations, and applies conservative heuristic repairs when printed values are unambiguous.
- **Cent-Exact Dual-Mode Bill Splitting:** Supports both equal item splitting and non-uniform custom weighting, distributing fractional paise/cents through the largest-remainder method without reconciliation loss.

2. System Architecture & End-to-End Pipeline

The system is constructed as a decoupled client-server architecture. The frontend is built on React 18, TypeScript, and Vite, deployable to Vercel edge networks. The backend is an asynchronous FastAPI service capable of CPU or CUDA execution.



2.1 Image Quality Gate & Preprocessing

Before invoking model inference, images pass through `scripts/image_quality.py`:

- **Blur Estimation:** Variance of the Laplacian operator. Images falling below empirical thresholds trigger a `retake_photo` status.
- **Dynamic Range & Luminance:** Checks histogram dispersion and average lightness to reject overexposed or illegibly dim inputs.
- **Preprocessing (OpenCV):** Normalizes orientation, applies Contrast Limited Adaptive Histogram Equalization (CLAHE), and formats inputs into `640x480` tensors.

2.2 Unified Receipt Data Contract

Extraction output conforms strictly to `config/receipt_schema.json`:

```
{
  "items": [
    { "name": "Paneer Butter Masala", "qty": 1, "unit_price": 220.0, "total": 220.0 },
    { "name": "Butter Naan", "qty": 3, "unit_price": 30.0, "total": 90.0 }
  ],
  "charges": [
    { "type": "CGST", "amount": 27.9 },
    { "type": "SGST", "amount": 27.9 },
    { "type": "Service Charge", "amount": 31.0 }
  ],
  "subtotal": 310.0,
  "grand_total": 396.8
}
```

3. Vision-Language Modeling (Donut V2)

Traditional document pipelines run an optical character recognition engine (e.g., Tesseract, EasyOCR) to generate text bounding boxes, followed by rule-based heuristic clustering or a secondary LayoutLM model. This approach frequently breaks on receipt crumpling, skewed thermal alignments, and tabular column drift.

3.1 Architecture Specifications

SplitSnap uses **Donut (Document Understanding Transformer)**:

- **Visual Encoder:** Swin Transformer backbone extracting hierarchical visual feature representations directly from receipt pixels.
- **Text Decoder:** Autoregressive multilingual BART architecture decoding tokens conditioned on the visual embeddings.
- **Vocabulary & Prompting:** Structured task tokens (`<s_receipt>...</s_receipt>`) prompt the decoder to emit deterministic JSON keys.

3.2 Dataset Pipeline & Indian Restaurant Augmentation

Model weights were fine-tuned across three distinct cohorts:

1. **CORD (Consolidated Receipt Dataset):** Comprehensive baseline for itemization and hierarchical line structures.
2. **SROIE:** High-density total, date, and vendor recognition.
3. **Synthetic Indian Restaurant Corpus (1,000+ Records):** Generated with localized nomenclature (Biryani, Tandoori, Thali, Paneer, Roti), realistic CGST (2.5%/9%), SGST, VAT, and varied service charge layouts. Geometric transformations include perspective warping, Gaussian blur, and simulated thermal paper fading.

Model Checkpoint	Training Data	Subtotal Accuracy	Item Recall	Grand Total F1
Baseline Donut (CORD)	CORD only	78.4%	74.2%	81.1%
Receipt Extractor V1	CORD + SROIE	84.1%	80.8%	87.3%
Receipt Extractor V2 (Active)	**CORD + SROIE + Synthetic Indian**	**93.6%**	**89.5%**	**95.2%**

4. Deterministic Bill Splitting & Custom Ratio Engine

The core split engine (`scripts/billing/split_bill.py`) solves the mathematical challenge of apportioning shared expenses, service charges, discounts, and fractional rounding.

4.1 Mathematical Formulation

Let N be the set of participants. For each line item $i \in I$ with total cost C_i:

1. **Participant Item Weights:** Each item has an assignment vector $W_i = \{w_{i,p} \mid p \in N\}$. If item i is split equally, $w_{i,p} = 1$ for assigned participants. If custom weighted, $w_{i,p} \ge 0$. The individual item share is:

```
s(i, p) = C_i * ( w_{i,p} / SUM_{q in N}(w_{i,q}) )
```

2. **Proportional Charge & Tax Allocation:** For external charges, service fees, or taxes $T = \sum \text{charges}$, and total subtotal $S = \sum_{p \in N} \text{item_total}(p)$:

```
charge_share(p) = T * ( item_total(p) / S )
```

3. **Hamilton-Chernoff (Largest Remainder) Cent Reconciliation:** All currency allocations are computed as integers (cents/paise). The integer floors are initially assigned: $\lfloor \text{amount} \times 100 \rfloor$. The remaining fractional units $\Delta = \text{Target} - \sum \lfloor \text{shares} \rfloor$ are sorted by descending fractional remainder and distributed one unit at a time. This guarantees that:

```
SUM_{p in N}(final_owed(p)) == Grand Total  (Exact to 0.00)
```

> **Guaranteed Invariants**
>
> The bill allocation engine guarantees zero unaccounted residual funds, monotonic non-negativity, and strict proportionality between consumed item totals and tax obligations.

5. Deployment, Portability & Future Roadmap

- **Hardware Footprint:** In CPU mode, the application runs comfortably on systems with **512 MB to 1 GB RAM**. With CUDA acceleration, complete inference completes in <3.0 seconds.
- **Extensibility Roadmap:** UPI Intent Link generation (PhonePe/GPay/Paytm), formatted WhatsApp clipboard exports, multi-receipt session merges, and WebSocket multi-device collaborative item claiming.